BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS prioritises nodes purely by g(n), the cost of the path already travelled from the start - it is uninformed and spreads out evenly in all directions. A* prioritises nodes by f(n) = g(n) + h(n), combining the cost so far with the heuristic estimate of the remaining cost to the goal. Because of the h(n) term, A* is guided toward the goal and expands far fewer nodes than UCS while still returning an optimal path (when h is admissible).

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

On this grid the agent can only move Up/Down/Left/Right, one cell per step at unit cost. Any path must change the x-coordinate |x1-x2| times and the y-coordinate |y1-y2| times, so no path can ever be shorter than the Manhattan distance - h(n) never overestimates the true remaining cost, which is exactly the definition of an admissible (and consistent) heuristic. Admissibility is what guarantees A* returns the optimal path the first time it pops the goal. If the heuristic were NOT admissible (it overestimated), A* could be biased away from the truly cheapest branch - a node on the optimal path could look expensive (high f) and be popped late or never, so the first goal reached could be via a suboptimal path: optimality would be lost, although A* would probably expand fewer nodes and run faster.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No. With 8-way movement a single diagonal step moves (1,1) at a cost of 1, but the Manhattan distance between those two cells is 2, so h(n) would overestimate the true remaining cost - Manhattan is no longer admissible. The correct metric is the Chebyshev distance, h(n) = max(|x1-x2|, |y1-y2|), which is admissible for 8-way movement with unit-cost diagonal steps; if diagonal moves cost sqrt(2) instead, the Octile distance max(dx,dy) + (sqrt(2)-1)*min(dx,dy) should be used. My euclidean_distance is also admissible in both cases, since the straight-line distance never exceeds any path cost, but it is weaker (less informed) than Chebyshev/Octile on a grid.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Eating all the remaining food is essentially a Travelling Salesman Problem over the pellets, so the heuristic should estimate the cost of the whole remaining tour, not just one hop. A strong admissible choice is the weight of a Minimum Spanning Tree (MST) over the set {agent position} united with all remaining food positions, using Manhattan distances as edge weights: the optimal collection route is a path that spans all those nodes, and an MST is a lower bound on any such path, so it never overestimates. Cheaper-to-compute alternatives that are still admissible include the maximum Manhattan distance from the agent to any remaining pellet, or the sum of distances along the nearest-neighbour chain of pellets (an optimistic estimate of the tour). Compared with "distance to the closest food", these consider every remaining pellet at once, so A* is steered toward the globally efficient eating order instead of greedy nearest-first behaviour.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

FACULTY OF COMPUTING